Policy Gradient

- So far, value-based R.L.

  $\Rightarrow$ near-deterministic policies

- Now: direct policy search

  $\Rightarrow$ stochastic policies.

Agenda

- Review on value-based RL
- Value-based vs Policy-based RL.
  - Why Stochastic Policy ?
- Policy Objective Function/Optimization
  - Policy Gradient Theorem.
  - Score function
- Algorithm : REINFORCE

Review on value-based RL.

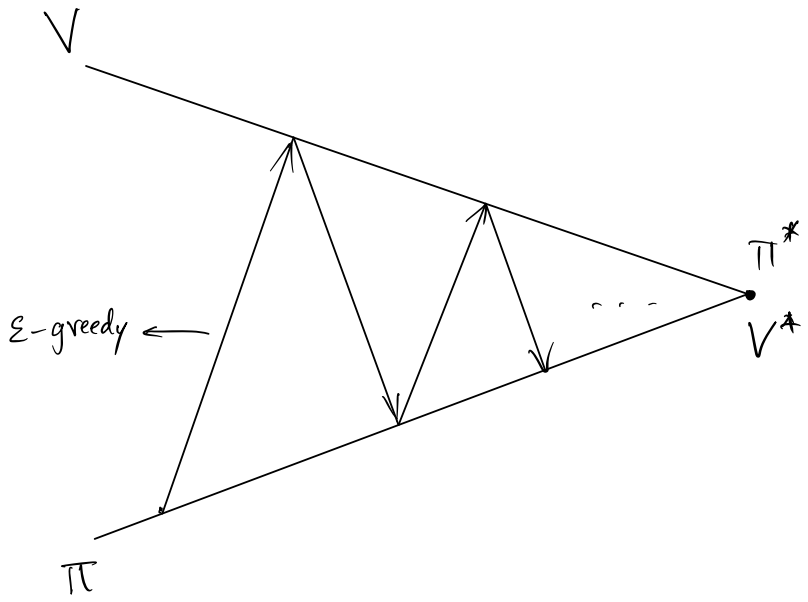
- Prediction / evaluation
  - Exact: $v_\pi(s)$ or $q_\pi(s, a)$
  - Approx: $\begin{cases} v(s, w) \approx v_\pi(s) \\ q(s, a, w) \approx q_\pi(s, a) \end{cases}$

- Improvement

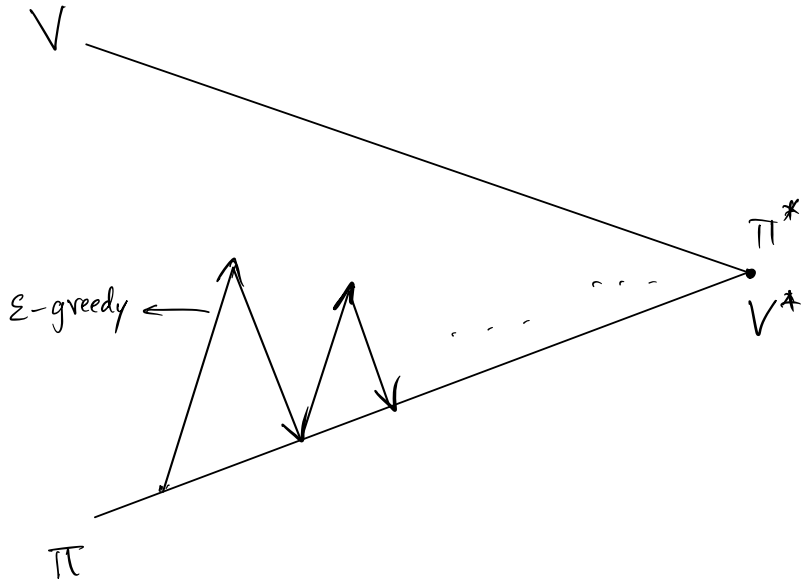
  A policy was generated/improved indirectly from the value functions

  e.g. using ε-greedy policy

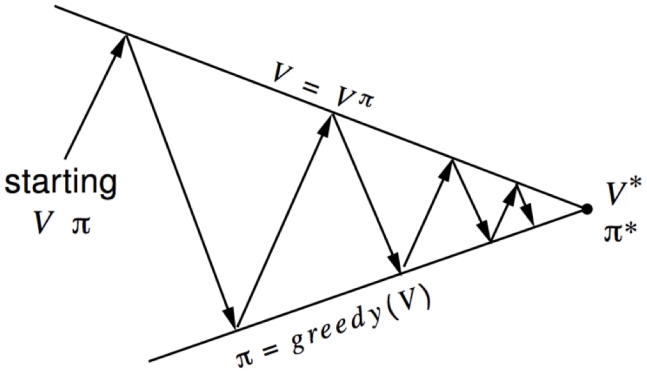
- Iterative

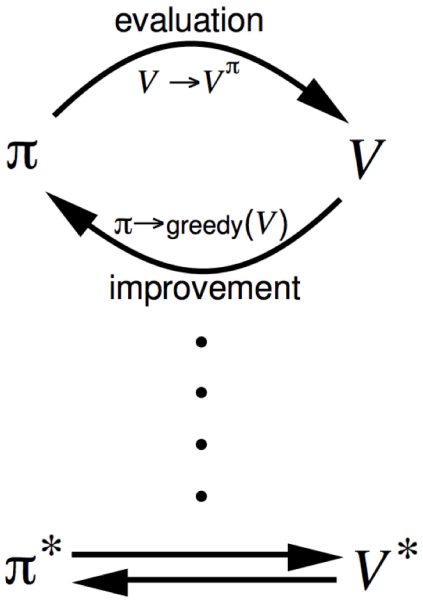


- Every time-step.



TD Learning



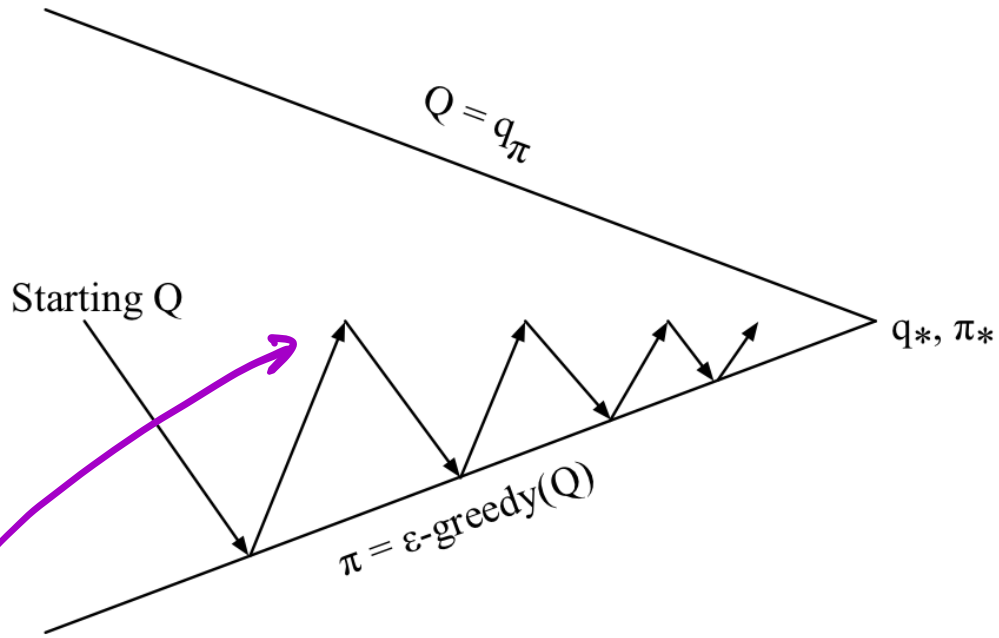


Policy evaluation Estimate v_π
e.g. Iterative policy evaluation

Policy improvement Generate $\pi' \geq \pi$
e.g. Greedy policy improvement

TD

On-policy control with SARSA



Every time-step:

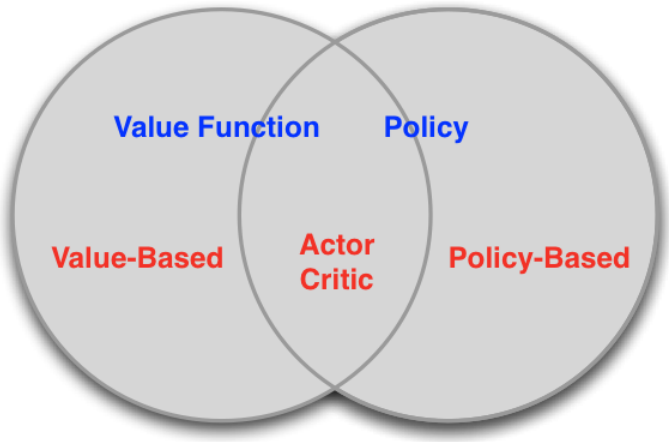
Policy evaluation Sarsa, $Q \approx q_\pi$

Policy improvement ϵ-greedy policy improvement

Today: Policy-based RL.

- We learn policy $\pi(a|s) = P(a|S_t = s)$

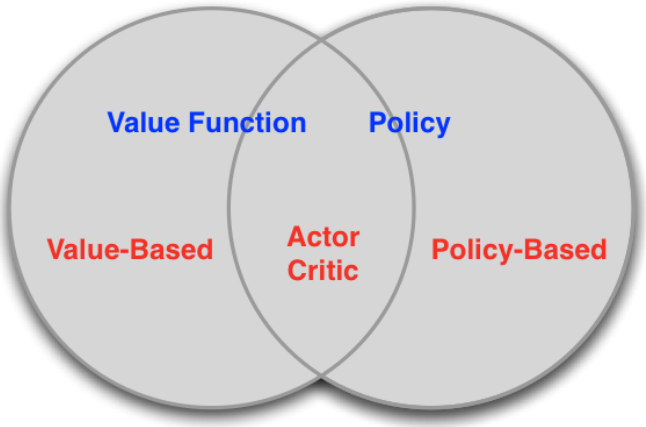
  or $\pi(a|s, \theta) = P(a|S_t = s, \theta)$ directly.



- Focus on Model-free RL.

Value-based vs Policy-based RL

	Learn Value Function	Learn Policy
Value-based	✓	Implicit (e.g. ε-greedy)
Policy-based	×	✓
Actor-Critic (Policy + Value)	✓	✓

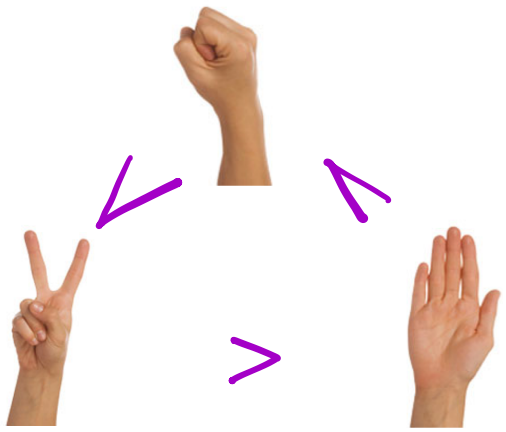


Adantages / dis-advantages

	Value-based	Policy-based	Actor-Critic
Stochastic Policy	×	✓	✓
Global/Local Optimum	Global	Local	Local
Effectiveness in handling high-dim action space	×	✓	✓

Why Stochastic Policy?

eg. 1. Rock - Paper - Scissors



- 2-player game : $R > S > P > R$
- State :
  history of prior actions (R, P, S)
  and if won or lost
- Reward :

$$R_t = \begin{cases} +1 & \text{Won} \\ -1 & \text{Lost} \end{cases}$$

- Q : Is deterministic policy optimal ?

- Policy :
  - A deterministic policy is easily exploited.
  - Optimal policy ?

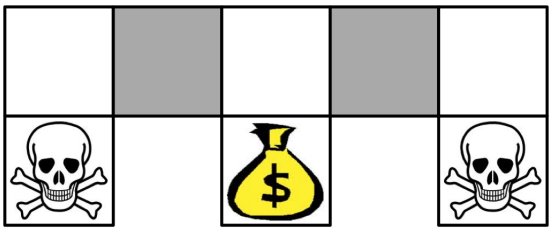
    Uniform Random Policy !

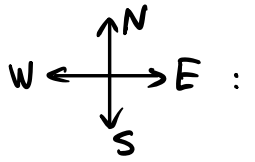
    (Nash equilibrium).

    ↓

    Easier said than done !!!

e.g. Aliased Gridworld



- States: $s_1, s_2, \cdots, s_8$. ($s_7 = s_8 = \otimes$).
- Actions: 

: E, N, W, N.
- Aliased: 2 grey grids are indistinguishable!



- Consider features of following form

$$\phi(s, a) = I\{s\}\ I\{a\}.$$

for s = Wall to N, Wall to S, Wall to E, Wall to W, Wall to N and S, etc.

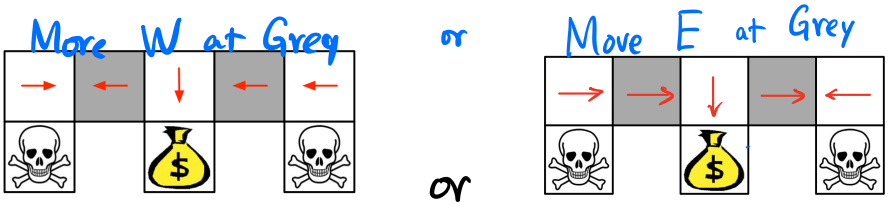
a = E, N, W, N.

- Value-based RL:

$$q(s,a;w) = f(\phi(s,a), w)$$

$\Rightarrow$ Near-Optimal policy under aliasing

① Greedy search:



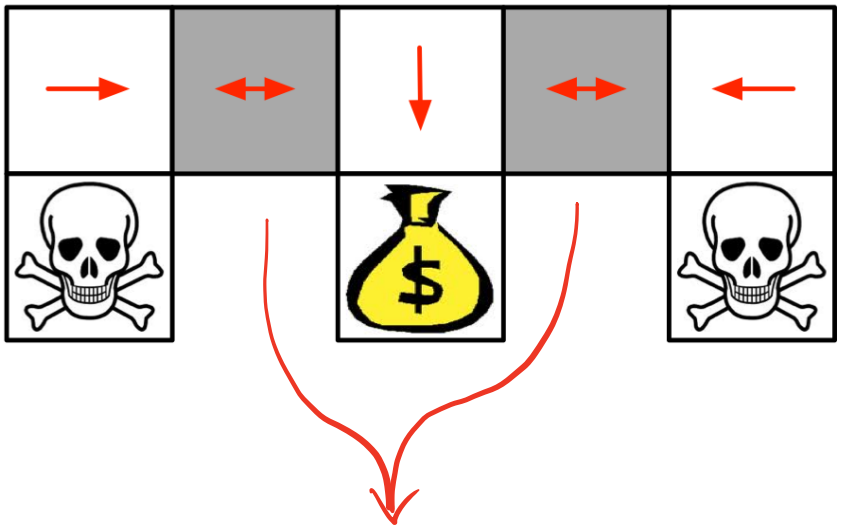
$\Rightarrow$ Could get stuck (never reach $)

② ε-greedy:

It will traverse the corridor for a long time.

- Policy-based R.L.:

Optimal Stochastic policy



$P(E \mid \text{Wall to S and N}) = \frac{1}{2}$, $P(W \mid \text{Wall to S and N}) = \frac{1}{2}$

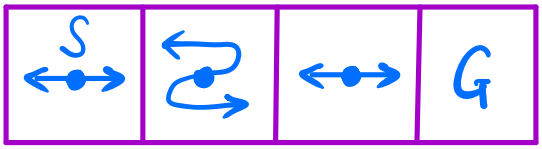
$\Rightarrow$

① Can learn optimal stochastic policy.

② Better convergence property:

Reach goal in a few steps with high prob.

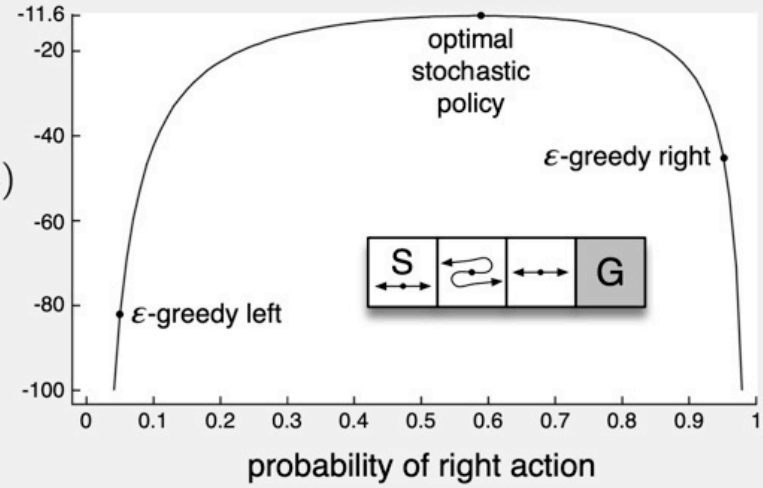
e.g. Short corridor with switched actions



Example 13.1 Short corridor with switched actions

Consider the small corridor gridworld shown inset in the graph below. The reward is -1 per step, as usual. In each of the three nonterminal states there are only two actions, right and left. These actions have their usual consequences in the first and third states (left causes no movement in the first state), but in the second state they are reversed, so that right moves to the left and left moves to the right. The problem is difficult because all the states appear identical under the function approximation. In particular, we define $\mathbf{x}(s, \text{right}) = [1, 0]^\top$ and $\mathbf{x}(s, \text{left}) = [0, 1]^\top$, for all s. An action-value method with ε-greedy action selection is forced to choose between just two policies: choosing right with high probability $1 - \varepsilon/2$ on all steps or choosing left with the same high probability on all time steps. If $\varepsilon = 0.1$, then these two policies achieve a value (at the start state) of less than -44 and -82, respectively, as shown in the graph. A method can do significantly better if it can learn a specific probability with which to select right. The best probability is about 0.59, which achieves a value of about -11.6.

$J(\boldsymbol{\theta}) = v_{\pi_{\boldsymbol{\theta}}}(\text{S})$



Policy Objective Functions

Goal: find "best" $\pi(a|s;\theta)$?

↑

How to measure ?

- Episodic environment:

$$J(\theta) = v_{\pi_\theta}(s_0) = \mathbb{E}_{\pi_\theta}(G_t \mid S_t = s_0)$$

- Continuing environment:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[v(S_t)] = \begin{cases} \int \mu(s)\, v_{\pi_\theta}(s)\, ds \\ \sum_{s \in S} \mu(s)\, v_{\pi_\theta}(s) \end{cases}$$

where $\mu(s)$: stationary distribution of Markov chain for π_θ.

- Focus: episodic case.

Easily extend to continuing/infinite horizon case

Policy Optimization

- Gradient-based
- Non-gradient-based

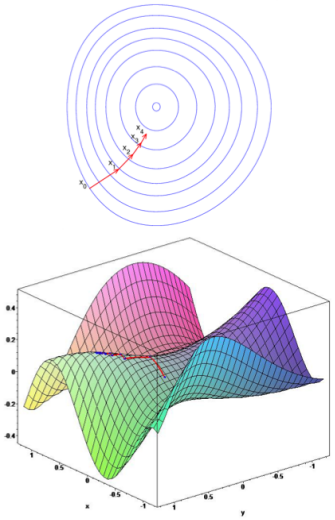
Policy Gradient

- Objective $\max_{\theta} J(\theta) = \mathbb{E}_{\pi_\theta} v(s_0)$
- Gradient Ascent:

$$\Delta\theta = \alpha \nabla_\theta J(\theta)$$

where

$$\nabla_\theta J(\theta) = \begin{pmatrix} \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{pmatrix}$$



Policy Gradient Theorem

Theorem: Under policy $\pi(\cdot|s;\theta)$, we have

$$\nabla_\theta J(\theta) = C \sum_s \mu(s) \sum_a q_\pi(s,a) \nabla \pi(a|s;\theta)$$

where

$$\mu(s) = \frac{\eta(s)}{\sum_{s'} \eta(s')} = \Pr\left(\text{visiting } s \text{ under policy } \pi, \text{ starting from } s_0\right)$$

(on-policy distribution under π)

$$\eta(s) = \sum_{k=0}^{\infty} \Pr(s_0 \to s, k, \pi) = \Pr\left(\text{moving from } s_0 \text{ to } s \text{ under policy } \pi\right)$$

($\Pr(s_0 \to s, k, \pi)$: Prob of moving from s_0 to s in k steps under policy π)

$$C = \begin{cases} \sum_{s'} \mu(s') = \text{average length of an episode.} & \text{if episodic} \\ 1 & \text{if continuing.} \end{cases}$$

#

Remark: $\nabla J(\theta)$ only involve derivative of π #

Corollary: Under policy $\pi(\cdot|s;\theta)$,

$$\nabla_\theta J(\theta) = C \sum_s \mu(s) \sum_a q_{\pi_\theta}(s,a) \nabla \pi(a|s;\theta)$$

$$= E_{\pi_\theta}\left[\sum_a q_{\pi_\theta}(s,a) \nabla \pi(a|s;\theta) \right]$$

Proof: If π is followed, then states will be encounted in proportion to $\mu(s)$. #

Proof of the Policy Gradient Theorem (episodic case)

With just elementary calculus and re-arranging of terms, we can prove the policy gradient theorem from first principles. To keep the notation simple, we leave it implicit in all cases that π is a function of $\boldsymbol{\theta}$, and all gradients are also implicitly with respect to $\boldsymbol{\theta}$. First note that the gradient of the state-value function can be written in terms of the action-value function as

$$
\begin{aligned}
\nabla v_\pi(s) &= \nabla \left[\sum_a \pi(a|s) q_\pi(s,a) \right], \quad \text{for all } s \in \mathcal{S} && \text{(Exercise 3.18)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \nabla q_\pi(s,a) \Big] && \text{(product rule of calculus)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \nabla \sum_{s',r} p(s',r|s,a) \big(r + v_\pi(s')\big) \Big] \\
& && \text{(Exercise 3.19 and Equation 3.2)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \sum_{s'} p(s'|s,a) \nabla v_\pi(s') \Big] && \text{(Eq. 3.4)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \sum_{s'} p(s'|s,a) && \text{(unrolling)} \\
&\qquad \sum_{a'} \big[\nabla \pi(a'|s') q_\pi(s',a') + \pi(a'|s') \sum_{s''} p(s''|s',a') \nabla v_\pi(s'') \big] \Big] \\
&= \sum_{x \in \mathcal{S}} \sum_{k=0}^{\infty} \Pr(s \to x, k, \pi) \sum_a \nabla \pi(a|x) q_\pi(x,a), && \text{(*) (see below)}
\end{aligned}
$$

after repeated unrolling, where $\Pr(s \to x, k, \pi)$ is the probability of transitioning from state s to state x in k steps under policy π. It is then immediate that

$$
\begin{aligned}
\nabla J(\boldsymbol{\theta}) &= \nabla v_\pi(s_0) \\
&= \sum_s \left(\sum_{k=0}^{\infty} \Pr(s_0 \to s, k, \pi) \right) \sum_a \nabla \pi(a|s) q_\pi(s,a) \\
&= \sum_s \eta(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(box page 199)} \\
&= \sum_{s'} \eta(s') \sum_s \frac{\eta(s)}{\sum_{s'} \eta(s')} \sum_a \nabla \pi(a|s) q_\pi(s,a) \\
&= \sum_{s'} \eta(s') \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(Eq. 9.3)} \\
&\propto \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(Q.E.D.)}
\end{aligned}
$$

Constant

(*)

$$= \sum_a \nabla \pi(a|s) q_\pi(s,a)$$

$$+ \sum_{s'} \sum_a \pi(a|s) p(s'|s,a) \sum_{a'} \nabla \pi(a'|s') q_\pi(s',a')$$

$$\left(\sum_{s'} \Pr(S \to s', k=1, \pi)\right) \left(\sum_a \nabla \pi(a|s') q_\pi(s',a)\right)$$

$$+ \cdots \cdots$$

$$= \sum_a \nabla \pi(a|s) q_\pi(s,a)$$

$$+ \sum_x \Pr(S \to x, k=1, \pi) \sum_a \nabla \pi(a|x) q_\pi(x,a)$$

$$+ \cdots \cdots$$

$$= \sum_x \Pr(S \to x, k=0, \pi) \sum_a \nabla \pi(a|x) q_\pi(x,a) \qquad (x=s)$$

$$+ \sum_x \Pr(S \to x, k=1, \pi) \sum_a \nabla \pi(a|x) q_\pi(x,a)$$

$$+ \cdots \cdots$$

$$= \sum_x \sum_{k=0}^{\infty} \Pr(S \to x, k, \pi) \sum_a \nabla \pi(a|x) q_\pi(x,a)$$

approximate values differ from the true values and is often used in plots. Often $\mu(s)$ is chosen to be the fraction of time spent in s. Under on-policy training this is called the *on-policy distribution*; we focus entirely on this case in this chapter. In continuing tasks, the on-policy distribution is the stationary distribution under π.

The on-policy distribution in episodic tasks

In an episodic task, the on-policy distribution is a little different in that it depends on how the initial states of episodes are chosen. Let $h(s)$ denote the probability that an episode begins in each state s, and let $\eta(s)$ denote the number of time steps spent, on average, in state s in a single episode. Time is spent in a state s if episodes start in s, or if transitions are made into s from a preceding state $\bar{s}$ in which time is spent:

$$\eta(s) = h(s) + \sum_{\bar{s}} \eta(\bar{s}) \sum_{a} \pi(a|\bar{s}) p(s\,|\,\bar{s}, a), \quad \text{for all } s \in \mathcal{S}. \tag{9.2}$$

This system of equations can be solved for the expected number of visits $\eta(s)$. The on-policy distribution is then the fraction of time spent in each state normalized to sum to one:

$$\mu(s) = \frac{\eta(s)}{\sum_{s'} \eta(s')}, \quad \text{for all } s \in \mathcal{S}. \tag{9.3}$$

This is the natural choice without discounting. If there is discounting ($\gamma < 1$) it should be treated as a form of termination, which can be done simply by including a factor of γ in the second term of (9.2).

Summary on Policy Gradient Algorithms

- The policy gradient has many equivalent forms

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ G_t\right] && \text{REINFORCE} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ Q^w(s,a)\right] && \text{Q Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ A^w(s,a)\right] && \text{Advantage Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ \delta\right] && \text{TD Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ \delta e\right] && \text{TD}(\lambda)\text{ Actor-Critic} \\
G_\theta^{-1}\nabla_\theta J(\theta) &= w && \text{Natural Actor-Critic}
\end{aligned}
$$

- Each leads a stochastic gradient ascent algorithm
- Critic uses policy evaluation (e.g. MC or TD learning) to estimate $Q^\pi(s,a)$, $A^\pi(s,a)$ or $V^\pi(s)$

Score function

- $\nabla_\theta J(\theta)$ only involve derivative of π

- $$\nabla_\theta \pi(a|s;\theta)$$

$$= \pi(a|s;\theta) \frac{\nabla \pi(a|s;\theta)}{\pi(a|s;\theta)}$$

$$= \pi(a|s;\theta) \underbrace{\nabla \log \pi(a|s;\theta)}$$

"Sore function"

(derivative of log-likelihood)

Differentiable Policy Classes

- Softmax
- Gaussian
- Neural Networks

Softmax
(for discrete action spaces)

$$\pi(a|s;\theta) = \frac{e^{\phi(s,a)^T\theta}}{\sum_{a \in \mathcal{A}(s)} e^{\phi(s,a)^T\theta}}, \quad a \in \mathcal{A}(s)$$

$$\Rightarrow$$

$$\nabla_\theta \log \pi(a|s;\theta) = \phi(s,a) - \mathbb{E}_{\pi_\theta} \phi(s,A)$$

Proof: $\log \pi(a|s;\theta) = e^{\phi(s,a)^T\theta} - \log\left(\sum_{a \in \mathcal{A}(s)} e^{\phi(s,a)^T\theta}\right)$

$$\nabla_\theta \log \pi(a|s;\theta) = \phi(s,a) - \frac{\sum_{a \in \mathcal{A}(s)} \phi(s,a) e^{\phi(s,a)^T\theta}}{\sum_{a \in \mathcal{A}(s)} e^{\phi(s,a)^T\theta}}$$

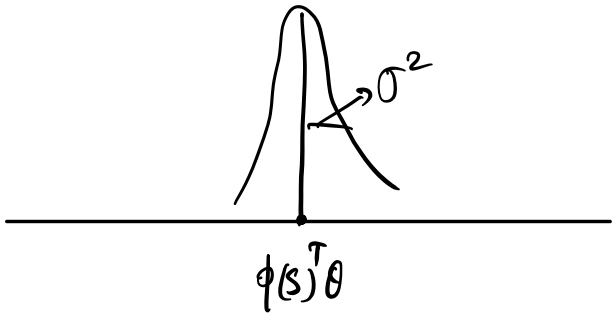
$$= \phi(s,a) - \mathbb{E}_{\pi_\theta} \phi(s,A) \quad \#$$

Gaussian Policy

(for continuous action spaces)

- $a \mid s \sim N(\phi(s)^T\theta, \sigma^2)$

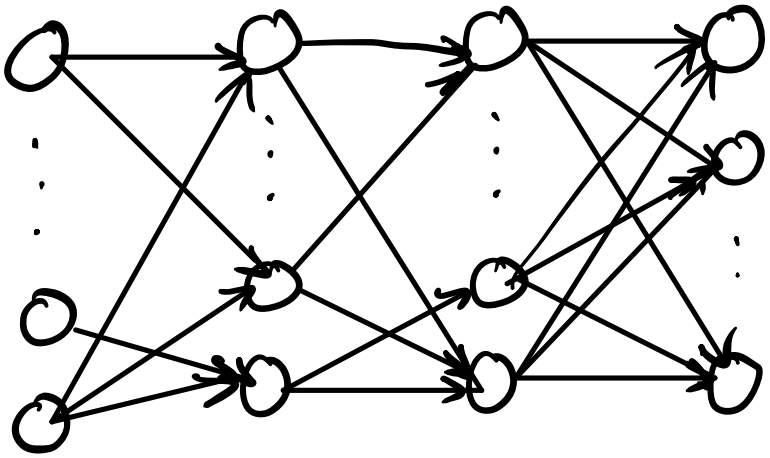
i.e $\pi(a \mid s; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(a - \phi(s)^T\theta)^2}$



- $\log \pi(a \mid s; \theta) = C - \frac{1}{2\sigma^2}(a - \phi(s)^T\theta)^2$

- $\nabla_\theta \log \pi(a \mid s; \theta) = \frac{(a - \phi(s)^T\theta)\,\phi(s)}{\sigma^2}$

Neural Networks



$s \xrightarrow{W} \pi(a|s)$

Algorithm: REINFORCE

Generic SGD

- Recall

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla \pi(a|S_t;\theta) \right]$$

- All-action SGD:

$$\Delta\theta = \alpha \sum_a \hat{q}_\pi(S_t, a) \nabla \pi(a|S_t;\theta)$$

$\hat{q}_\pi(S_t, a)$: An estimate of $q_\pi(S_t, a)$

$\sum_a$: all actions

NOT feasible

- Q: reduce "all actions" to current action A_t ?

- Solution :

① $\sum_a \simeq E_\pi$

② Sampling E_π :

$$\{S_0, A_0, R_1, S_1, \cdots S_{T-1}, A_{T-1}, R_{T-1}, S_T\}$$

$$\Downarrow$$

Online updating .

Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla \pi(a|S_t;\theta) \right]$$

$$= \mathbb{E}_\pi \left[q_\pi(S_t, A_t) \nabla \log \pi(A_t|S_t;\theta) \right]$$

$$= \mathbb{E}_\pi \left[G_t \frac{\nabla \pi(A_t|S_t;\theta)}{\pi(A_t|S_t;\theta)} \right]$$

$$= \mathbb{E}_\pi \left[G_t \nabla \log \pi(A_t|S_t;\theta) \right]$$

#

Proof

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla \pi(a|S_t;\theta) \right]$$

$$= \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \frac{\nabla \pi(a|S_t;\theta)}{\pi(a|S_t;\theta)} \pi(a|S_t;\theta) \right]$$

$$= \mathbb{E}_\pi \left[q_\pi(S_t, A_t) \frac{\nabla \pi(A_t|S_t;\theta)}{\pi(A_t|S_t;\theta)} \right]$$

$$= \mathbb{E}_\pi \left[\mathbb{E}_\pi(G_t | S_t, A_t) \frac{\nabla \pi(A_t|S_t;\theta)}{\pi(A_t|S_t;\theta)} \right]$$

$$= \mathbb{E}_\pi \left[\mathbb{E}_\pi \left(G_t \frac{\nabla \pi(A_t|S_t;\theta)}{\pi(A_t|S_t;\theta)} \middle| S_t, A_t \right) \right]$$

$$= \mathbb{E}_\pi \left[G_t \frac{\nabla \pi(A_t|S_t;\theta)}{\pi(A_t|S_t;\theta)} \right]$$

#

REINFORCE:

Monte-Carlo Policy Gradient:

$$\Delta\theta = \alpha G_t \frac{\nabla \pi(A_t|S_t;\theta)}{\pi(A_t|S_t;\theta)}$$

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\qquad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Remarks

- $\Delta\theta \propto G_t \times \dfrac{\text{Gradient of } \pi(A_t|S_t;\theta)}{\pi(A_t|S_t;\theta)}$

- Proportional to Gradient of $\pi(A_t|S_t;\theta)$:

  In the direction in parameter space that most increase prob of repeating action A_t on future visits to S_t.

- Inversely proportional to $\pi(A_t|S_t;\theta)$.

  Otherwise, more frequently visited states are at an advantage, even if they are not the best.

- Hence, it causes the parameters to move most in the direction that favors actions that yield the highest returns, and also prevents overfitting.

e.g. Short corridor gridworld

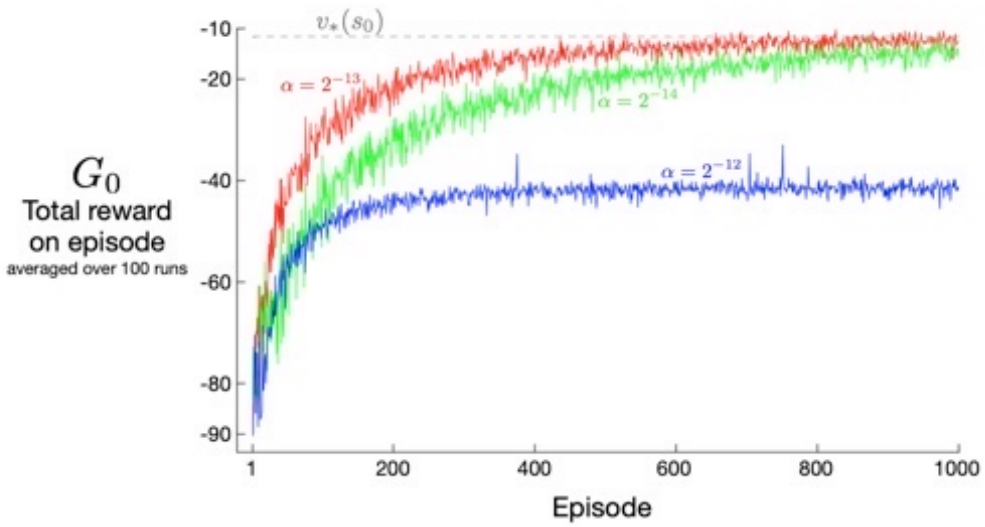


Figure 13.1: REINFORCE on the short-corridor gridworld (Example 13.1). With a good step size, the total reward per episode approaches the optimal value of the start state.

Drawbacks of REINFORCE

- REINFORCE = MC Policy Gradient

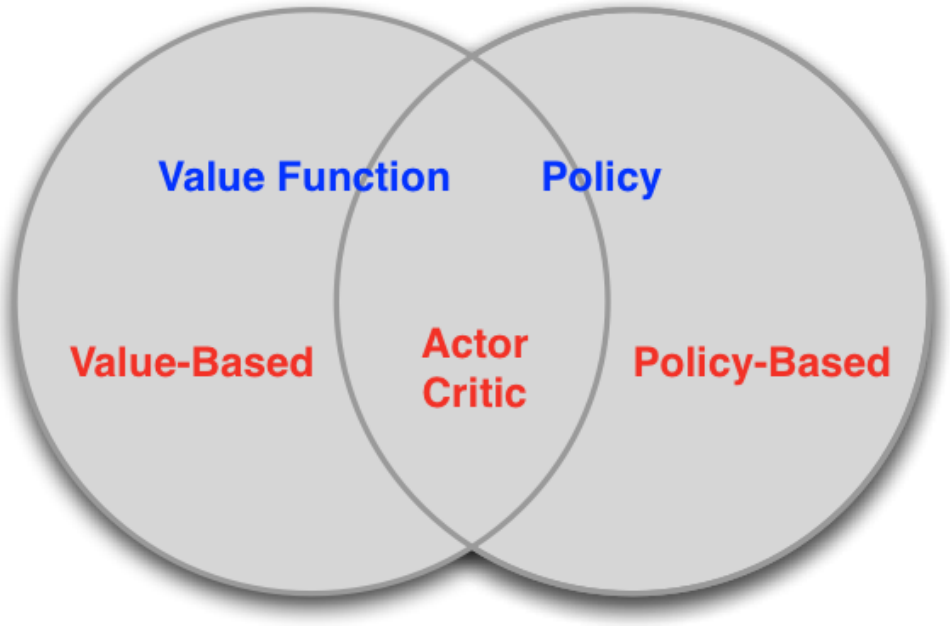
  high variance

  $\Longrightarrow$ slow learning.

- Variance reduction techiques:
  - Actor – Critic
  - Introduce a baseline.
  - Many others.

Actor-Critic

(An approximate policy gradient algorithm)



- Recall

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[q_\pi(S_t, A_t) \nabla \log \pi(A_t | S_t; \theta) \right]$$

- REINFORCE: Monte-Carlo Policy Gradient:

$$\Delta\theta = \alpha \, G_t \, \frac{\nabla \pi(A_t | S_t; \theta)}{\pi(A_t | S_t; \theta)}$$

G_t: self-learning

- May help with indep judge/critic to evaluate $q_\pi(S_t, A_t)$:

$$Q(s, a, w) \approx q_\pi(s, a).$$

- Actor-Critic algorithm:

$$\Delta\theta = \alpha \; Q(s, a, w) \; \nabla \log \pi(A_t | S_t; \theta)$$

- Two sets of parameters

  Critic : update action-value function parameter, w.

  Actor : update policy parameter, θ, suggested by critic.

- Variance reduction ?

  Evaluation (w) and Improvement (θ) are decoupled !

Estimating $q(s, a, w)$

- Monte-Carlo policy evaluation
- TD learning
- TD(λ)

e.g. TD(0).

Action-Value Actor-Critic

- Simple actor-critic algorithm based on action-value critic
- Using linear value fn approx. $Q_w(s,a) = \phi(s,a)^\top w$

  Critic Updates w by linear TD(0)

  Actor Updates θ by policy gradient

function QAC

  Initialise s, θ

  Sample $a \sim \pi_\theta$

  for each step **do**

    Sample reward $r = \mathcal{R}_s^a$; sample transition $s' \sim \mathcal{P}_{s,\cdot}^a$

    Sample action $a' \sim \pi_\theta(s', a')$

    $\delta = r + \gamma Q_w(s', a') - Q_w(s, a)$

    $\theta = \theta + \alpha \nabla_\theta \log \pi_\theta(s, a) Q_w(s, a)$

    $w \leftarrow w + \beta \delta \phi(s, a)$

    $a \leftarrow a', s \leftarrow s'$

  end for

end function

REINFORCE with a baseline

- **Theorem:**

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[q_\pi(S_t, A_t) \nabla \log \pi(A_t | S_t; \theta) \right]$$

$$= \mathbb{E}_\pi \left[\{ q_\pi(S_t, A_t) - B(S_t) \} \nabla \log \pi(A_t | S_t; \theta) \right]$$

#

- **A good choice of $B(s)$**

$$B(s) = V_\pi(s) = \mathbb{E} q_\pi(s, A_t) = \sum_a \pi(a|s) q_\pi(s, a)$$

- **Advantage function:**

$$A_\pi(s, a) = q_\pi(s, a) - V_\pi(s).$$

- **Proof**:

$$\mathbb{E}_\pi\Big[B(S_t)\,\nabla \log \pi(A_t|S_t;\theta)\Big]$$

$$= \mathbb{E}_\pi\Big\{\mathbb{E}_\pi\Big[B(S_t)\,\nabla \log \pi(A_t|S_t;\theta)\Big]\,\Big|\,S_t\Big\}$$

$$= \mathbb{E}_\pi\Big\{B(S_t)\,\underbrace{\mathbb{E}_\pi\Big[\nabla \log \pi(A_t|S_t;\theta)\Big]}_{=0}\,\Big|\,S_t\Big\}$$

$$= 0$$

Since $\mathbb{E}_\pi\Big[\nabla \log \pi(A_t|S_t;\theta)\Big] = \mathbb{E}\left[\dfrac{\nabla \pi(A_t|S_t;\theta)}{\pi(A_t|S_t;\theta)}\right]$

$$= \sum_a \left\{\pi(a|S_t;\theta) \times \frac{\nabla \pi(a|S_t;\theta)}{\pi(a|S_t;\theta)}\right\} = \sum \nabla \pi(a|S_t;\theta)$$

$$= \nabla \sum \pi(a|S_t;\theta) = \nabla 1 = 0 \quad . \quad \#$$

Effect of $A(s,a)$

- Advantage function $A(s,a)$ can significantly reduce variance.
- The Critic should really estimate Advantage $A(s,a)$. instead of $Q(s,a)$.

- Two function approximations with two different parameters.

$$V(s, v) \approx v_{\pi_\theta}(s)$$

$$Q(s, a, w) \approx q_{\pi_\theta}(s, a)$$

$$A(s, a) = Q(s, a, w) - V(s, v).$$

- Update w and v by TD(0), e.g.

An estimate of $A(s,a)$ by TD error

- Recall

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[A_{\pi_\theta}(s,a) \nabla \log \pi(A_t | s_t; \theta) \right]$$

- TD error

$$\delta_{\pi_\theta} = r + \gamma V_{\pi_\theta}(s') - V_{\pi_\theta}(s)$$

is unbiased for $A_{\pi_\theta}(s,a)$, i.e.

$$\mathbb{E}_{\pi_\theta}(\delta_{\pi_\theta} | s,a) = A_{\pi_\theta}(s,a) \quad (*)$$

- Proof of ⊛ :

$$\mathbb{E}_{\pi_\theta}(\delta_{\pi_\theta} | s, a)$$

$$= \mathbb{E}_{\pi_\theta}\left[r + \gamma V_{\pi_\theta}(s') - V_{\pi_\theta}(s) \,\middle|\, s, a \right]$$

$$= \mathbb{E}_{\pi_\theta}\left[r + \gamma V_{\pi_\theta}(s') \,\middle|\, s, a \right] - V_{\pi_\theta}(s)$$

$$= q_{\pi_\theta}(s, a) - V_{\pi_\theta}(s)$$

$$= A_{\pi_\theta}(s, a) \quad \#$$

- In practice, use

$$\delta_v = r + \gamma V(s', v) - V(s, v).$$

- Only one set of parameters, v, is needed.

Choice of Critics

- MC target

$$\Delta\theta = \alpha\left(G_t - V(s;\theta)\right)\phi(s)$$

- TD(0) target:

$$\Delta\theta = \alpha\left(r + \gamma V(s';\theta) - V(s;\theta)\right)\phi(s)$$

Choice of Actors

- Recall

$$\nabla J(\theta) = \mathbb{E}_{\pi_\theta}\left[A_{\pi_\theta}(s,a)\, \nabla_\theta \log \pi(a|s;\theta)\right]$$

- MC policy gradient:

$$\Delta\theta = \alpha\, A_{\pi_\theta}(s,a)\,\left(G_t - V(s;v)\right)$$

- TD(0) policy gradient:

$$\Delta\theta = \alpha\, A_{\pi_\theta}(s,a)\,\left(r + \gamma V(s';\theta) - V(s;\theta)\right)$$

#

Summary on Policy Gradient Algorithms

- The policy gradient has many equivalent forms

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ G_t\right] && \text{REINFORCE} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ Q^w(s,a)\right] && \text{Q Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ A^w(s,a)\right] && \text{Advantage Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ \delta\right] && \text{TD Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ \delta e\right] && \text{TD}(\lambda)\text{ Actor-Critic} \\
G_\theta^{-1}\nabla_\theta J(\theta) &= w && \text{Natural Actor-Critic}
\end{aligned}
$$

- Each leads a stochastic gradient ascent algorithm
- Critic uses policy evaluation (e.g. MC or TD learning) to estimate $Q^\pi(s,a)$, $A^\pi(s,a)$ or $V^\pi(s)$